

ใบงานหัวข้อที่ 14

แบบฝึกถ่ายโอนความคิด: School Club Recommendation

ระบบแนะนำชมรมด้วย Neo4j Graph Database (Cypher)

*Node → Relationship → Property → Pattern → CREATE → MATCH → WHERE → RETURN → Traversal →
Aggregation → Recommendation*

รายการ	รายละเอียด
ชื่อ - สกุล	ณรงค์ศักดิ์ ประเสริฐศิริสร
รหัสนักศึกษา	664245033 หมู่เรียน 66/44
สาขาวิชา	วิทยาการคอมพิวเตอร์ มหาวิทยาลัยราชภัฏนครปฐม
เครื่องมือที่ใช้	Neo4j Desktop / Neo4j Aura + Neo4j Browser (ภาษา Cypher)
วันที่ส่งงาน	

1. โจทย์และเป้าหมายของใบงาน

ใบงานนี้ให้เปลี่ยนโดเมนจาก “ระบบแนะนำหนังสือในโรงเรียน” ไปเป็น “ระบบแนะนำชมรม” เพื่อพิสูจน์ว่าเราเข้าใจสายความคิดของ Graph Database จริง ไม่ได้เพียงจำ Query เดิมมาใช้ซ้ำ งานที่ต้องทำมี 6 ข้อ ดังนี้

- กำหนด Node และ Relationship ที่จำเป็นสำหรับระบบชมรม
- สร้างข้อมูลตัวอย่างอย่างน้อย Student 5 คน และ Club 3 ชมรม
- ใช้ MATCH + WHERE + RETURN เพื่อดูชมรมของเพื่อน S001
- ใช้ Traversal ไปยัง Club และใช้ count(DISTINCT friend) ให้คะแนน
- ตัด Club ที่ S001 เป็น MEMBER_OF อยู่แล้วออก แล้วสร้าง Recommendation
- อธิบายเป็นภาษาไทย 2-3 ประโยคว่าทำไมระบบจึงแนะนำ Club อันดับ 1

1.1 การออกแบบ Node

ขั้นแรกคือหา “สิ่งที่ควรมีตัวตนของตนเอง” ในระบบ โดยขีดเส้นใต้คำนามสำคัญในโจทย์ ได้แก่ นักเรียน ชมรม และความสนใจ

Node Label	ความหมาย	Property ที่เก็บ
:Student	นักเรียนในโรงเรียน	student_id, name, grade
:Club	ชมรมที่เปิดให้สมัคร	club_id, name, room
:Interest	หัวข้อความสนใจ	interest_name

1.2 การออกแบบ Relationship

ขั้นต่อมาคือเปลี่ยน “คำกริยา” ในโจทย์ให้เป็นความสัมพันธ์ เช่น เป็นเพื่อนกับ เป็นสมาชิกของ สนใจเรื่อง และเน้นด้าน

ความหมายในโลกจริง	Relationship Type	Pattern
นักเรียนเป็นเพื่อนกัน	FRIEND_OF	(s:Student)-[:FRIEND_OF]-(f:Student)
นักเรียนเป็นสมาชิกชมรม	MEMBER_OF	(s:Student)-[:MEMBER_OF]->(c:Club)
นักเรียนสนใจเรื่องใด	INTERESTED_IN	(s:Student)-[:INTERESTED_IN]->(i:Interest)
ชมรมเน้นด้านใด	FOCUSES_ON	(c:Club)-[:FOCUSES_ON]->(i:Interest)

2. ขั้นที่ 1 — สร้าง Constraint ของตัวระบุ

Constraint ทำหน้าที่บังคับว่าตัวระบุห้ามซ้ำ ช่วยกันไม่ให้เผลอสร้างนักเรียนรหัสเดียวกันสองครั้ง และยังช่วยให้ Neo4j ค้นหาด้วยรหัสได้เร็วขึ้น ให้รันทีละคำสั่ง

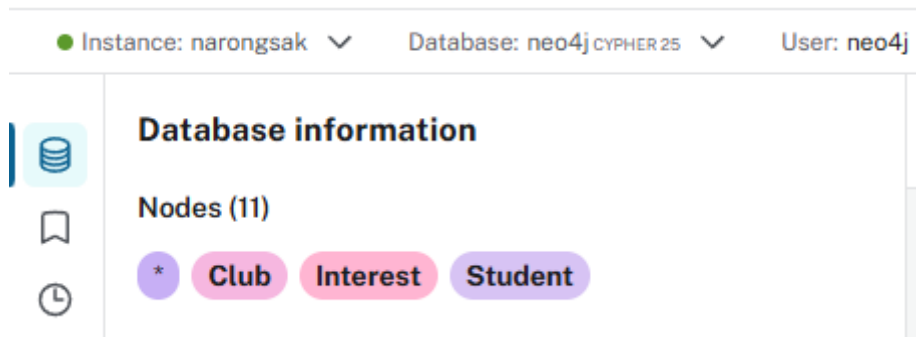
```
CREATE CONSTRAINT student_id_unique IF NOT EXISTS
FOR (s:Student) REQUIRE s.student_id IS UNIQUE;
```

```
CREATE CONSTRAINT club_id_unique IF NOT EXISTS
FOR (c:Club) REQUIRE c.club_id IS UNIQUE;
```

```
CREATE CONSTRAINT interest_name_unique IF NOT EXISTS
FOR (i:Interest) REQUIRE i.interest_name IS UNIQUE;
```

ตรวจสอบว่าสร้างครบหรือไม่ ด้วยคำสั่ง

```
SHOW CONSTRAINTS;
```



3. ขั้นที่ 2 — CREATE ข้อมูลตัวอย่าง

สร้างนักเรียน 5 คน ชมรม 3 ชมรม และความสนใจ 3 หัวข้อ พร้อมความสัมพันธ์ทั้งหมดในคำสั่งเดียว

ข้อมูลชุดนี้ออกแบบมาให้เห็นผลของทุกขั้นตอนอย่างชัดเจน

```
CREATE

(s1:Student {student_id:'S001', name:'Anan',   grade:'M.6'}),
(s2:Student {student_id:'S002', name:'Mali',   grade:'M.6'}),
(s3:Student {student_id:'S003', name:'Somchai', grade:'M.5'}),
(s4:Student {student_id:'S004', name:'Nida',   grade:'M.6'}),
(s5:Student {student_id:'S005', name:'Pim',    grade:'M.5'}),

(c1:Club {club_id:'CL01', name:'Computer Club',   room:'IT-101'}),
(c2:Club {club_id:'CL02', name:'Music Club',      room:'MU-201'}),
(c3:Club {club_id:'CL03', name:'Photography Club', room:'AR-301'}),

(i1:Interest {interest_name:'Programming'}),
(i2:Interest {interest_name:'Music'}),
(i3:Interest {interest_name:'Photography'}),

// S001 เป็นเพื่อนกับ S002, S003, S004 แต่ไม่ได้เป็นเพื่อนกับ S005
(s1)-[:FRIEND_OF]->(s2),
(s1)-[:FRIEND_OF]->(s3),
(s1)-[:FRIEND_OF]->(s4),

// การเป็นสมาชิกชมรม พร้อมวันที่สมัคร
(s1)-[:MEMBER_OF {joined_date:date("2026-05-20")}]>(c1),
(s2)-[:MEMBER_OF {joined_date:date("2026-05-21")}]>(c1),
(s2)-[:MEMBER_OF {joined_date:date("2026-05-22")}]>(c2),
(s3)-[:MEMBER_OF {joined_date:date("2026-05-23")}]>(c2),
(s3)-[:MEMBER_OF {joined_date:date("2026-05-24")}]>(c3),
(s4)-[:MEMBER_OF {joined_date:date("2026-05-25")}]>(c2),
(s5)-[:MEMBER_OF {joined_date:date("2026-05-26")}]>(c3),

// ความสนใจของนักเรียน และด้านที่แต่ละชมรมเน้น
(s1)-[:INTERESTED_IN]->(i1),
(s1)-[:INTERESTED_IN]->(i2),
(c1)-[:FOCUSES_ON]->(i1),
(c2)-[:FOCUSES_ON]->(i2),
(c3)-[:FOCUSES_ON]->(i3);
```

Instance: narongsak Database: neo4j CYPHER 25 User: neo4j

Database information

Nodes (11)

- * Club Interest Student

Relationships (15)

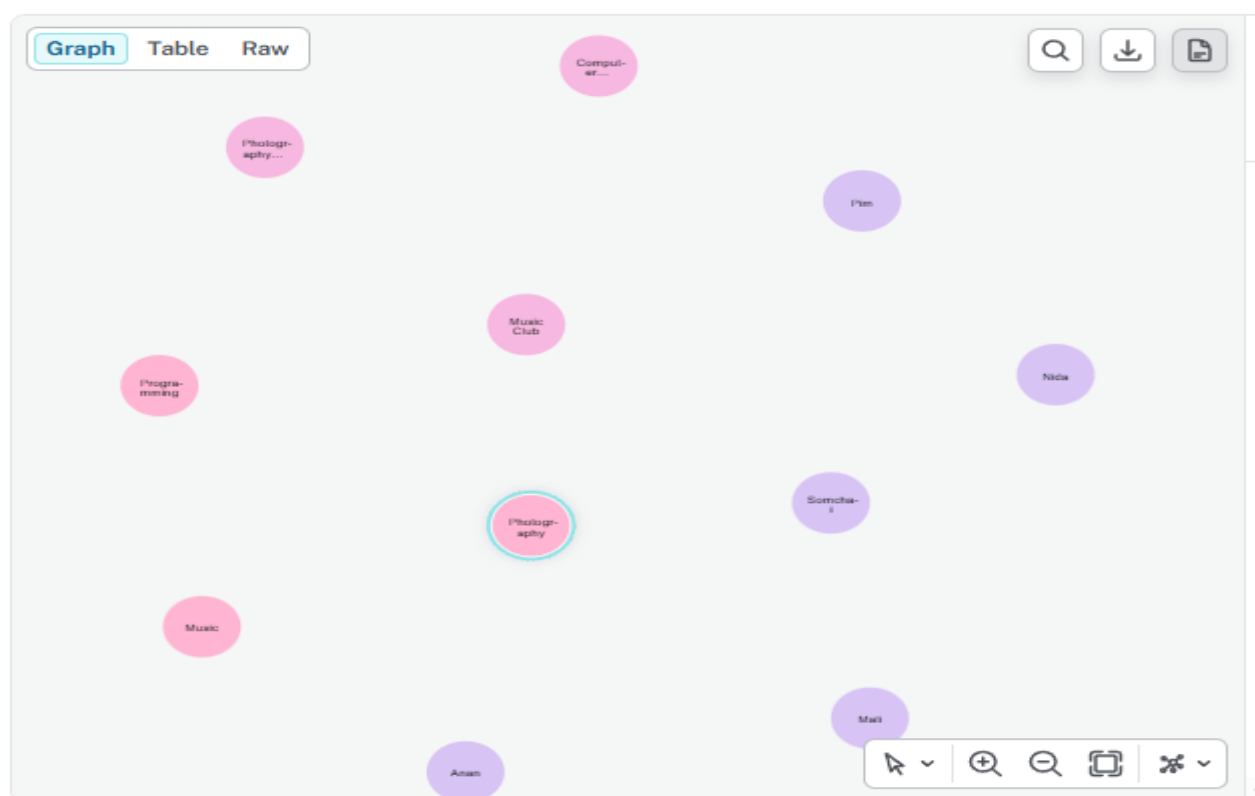
- * FOCUSES_ON FRIEND_OF INTERESTED_IN MEMBER_OF

Property keys

- club_id grade interest_name joined_date name room student_id

3.1ดูภาพรวมกราฟที่สร้างได้

```
MATCH (n) RETURN n;
```



4. ขั้นที่ 3 — MATCH + WHERE + RETURN: ดูชมรมของเพื่อน S001

อ่านเป็นประโยคว่า “นักเรียน me เชื่อมกับนักเรียน friend ด้วย FRIEND_OF และ friend เชื่อมไปยัง club ด้วย MEMBER_OF”
เมื่ออ่าน Pattern เป็นประโยคได้แล้ว จึงแปลงเป็น Cypher

```
MATCH (me:Student)-[:FRIEND_OF]-(friend:Student)-[m:MEMBER_OF]->(club:Club)
WHERE me.student_id = 'S001'
RETURN friend.name AS friend,
       club.name AS club,
       m.joined_date AS joined_on
ORDER BY friend, club;
```

อธิบายทีละส่วน: MATCH บอกว่าต้องการโครงสร้างแบบใด (เพื่อนของฉันเป็นสมาชิกชมรมอะไร), WHERE จำกัดให้จุดเริ่มต้นเป็น S001 เท่านั้น, RETURN เลือกว่าจะแสดงอะไรออกมา และสังเกตว่าเราตั้งตัวแปร m ไว้บนเส้น MEMBER_OF จึงดึง Property joined_date ของเส้นนั้นมาแสดงได้

	friend	club	joined_on
1	"Mali"	"Computer Club"	2026-05-21
2	"Mali"	"Music Club"	2026-05-22
3	"Nida"	"Music Club"	2026-05-25
4	"Somchai"	"Music Club"	2026-05-23
5	"Somchai"	"Photography Club"	2026-05-24

5. ขั้นที่ 4 — Traversal + Aggregation: ให้คะแนนชมรม

Traversal คือการเดินทางจาก Node หนึ่งไปอีก Node หนึ่งตาม Relationship โดยนับเป็น Hop ละหนึ่งเส้น

Hop	เส้นทาง	ความหมาย
0	S001	จุดเริ่มต้น คือตัวเราเอง
1	S001 - FRIEND_OF - Friend	หาว่าเพื่อนของเราคือใคร
2	Friend - MEMBER_OF - Club	หาว่าเพื่อนอยู่ชมรมอะไร

นับจำนวนเพื่อนที่อยู่ในแต่ละชมรม

```
MATCH (me:Student {student_id:'S001'})
  -[:FRIEND_OF]-(friend:Student)
  -[:MEMBER_OF]->(club:Club)
RETURN club.club_id AS club_id,
       club.name AS club,
       count(DISTINCT friend) AS friend_score
ORDER BY friend_score DESC, club;
```

อธิบาย: count(DISTINCT friend) เปลี่ยน “รายการความสัมพันธ์” ให้กลายเป็น “คะแนน”

โดยนับจำนวนเพื่อนที่ไม่ซ้ำกันในแต่ละชมรม ส่วน DISTINCT มีไว้กันกรณีที่เพื่อนคนเดียวกันมีเส้น MEMBER_OF

ไปยังชมรมเดียวกันหลายเส้น ซึ่งจะทำให้นับซ้ำ

neo4j\$ MATCH (me:Student {student_id:'S001'}) -[:FRIEND_OF]-(friend:Student) -[:MEMBER_OF]->(club:Cl		
Table Raw		
club_id	club	friend_score
1 "CL02"	"Music Club"	3
2 "CL01"	"Computer Club"	1
3 "CL03"	"Photography Club"	1

6. ขั้นที่ 5 — Recommendation: ตัดชมรมที่เป็นสมาชิกแล้วออก

คะแนนในขั้นที่แล้วยังใช้แนะนำไม่ได้ เพราะ Computer Club ติดอันดับมาด้วย ทั้งที่ S001 เป็นสมาชิกอยู่แล้ว การแนะนำสิ่งที่มีอยู่แล้วถือว่าไม่มีประโยชน์ จึงต้องเพิ่มเงื่อนไขตัดออก

```
MATCH (me:Student {student_id:'S001'})
  -[:FRIEND_OF]-(friend:Student)
  -[:MEMBER_OF]->(club:Club)
WHERE NOT EXISTS {
  MATCH (me)-[:MEMBER_OF]->(club)
}
RETURN club.club_id AS club_id,
       club.name    AS club,
       count(DISTINCT friend) AS friend_score
ORDER BY friend_score DESC, club;
```

อธิบาย: NOT EXISTS { ... } คือ Subquery ที่ถามว่า “มีเส้น MEMBER_OF จาก me ไปยัง club นี้อยู่หรือไม่”

ถ้ามีแปลว่าเป็นสมาชิกอยู่แล้ว ให้ตัดแถวนั้นทิ้ง ผลคือ Computer Club หายไปจากผลลัพธ์ เหลือเฉพาะชมรมที่เป็น Candidate จริง ๆ

Table Raw		Q O ↓	
club_id	club	friend_score	
1 "CL02"	"Music Club"	3	
2 "CL03"	"Photography Club"	1	

Started streaming 2 records after 156 ms and completed after 182 ms.

โจทย์ระบุ Node :Interest และความสัมพันธ์ INTERESTED_IN กับ FOCUSES_ON ไว้ด้วย

จึงลองสร้างเส้นทางการแนะนำอีกแบบหนึ่งที่ไม่ผ่านเพื่อน แต่เดินทางความสนใจของตัวเองแทน

```
MATCH (me:Student {student_id:'S001'})
  -[:INTERESTED_IN]->(i:Interest)
  <-[:FOCUSES_ON]-(club:Club)
WHERE NOT EXISTS {
  MATCH (me)-[:MEMBER_OF]->(club)
}
RETURN club.name AS club,
       i.interest_name AS matched_interest;
```

S001 สนใจทั้ง Programming และ Music แต่ชมรมที่เน้น Programming คือ Computer Club ซึ่งเป็นสมาชิกอยู่แล้วจึงถูกตัดออก เหลือเพียง Music Club ผลลัพธ์นี้ตรงกับอันดับ 1 ที่ได้จากเส้นทางเพื่อน เป็นการยืนยันคำตอบจากหลักฐานคนละชุดกัน

Table

Raw

Q

O

Download

club	matched_interest
1 "Music Club"	"Music"

Started streaming 1 record after 85 ms and completed after 87 ms.

7. ขั้นที่ 6 — อธิบายเหตุผลของการแนะนำ

ระบบแนะนำ Music Club เป็นอันดับหนึ่ง เพราะเพื่อนของ S001 ทั้งสามคน คือ Mali, Somchai และ Nida อยู่ชมรมนี้ทั้งหมด พอนับออกมาเป็นคะแนนก็ได้ 3 ในขณะที่ Photography Club มีเพื่อนอยู่แค่คนเดียว ส่วน Computer Club ถึงแม้จะมีเพื่อนอยู่ด้วยก็ถูกตัดทิ้งไป เพราะ S001 เป็นสมาชิกอยู่แล้ว แนะนำไปก็ไม่ได้อะไรขึ้นมา สุดท้ายชมรมที่ขึ้นมาอันดับหนึ่งจึงต้องเป็นชมรมที่เพื่อนอยู่กันเยอะ และตัวเรายังไม่เคยเข้า

